Reporte Técnico de Actividades Práctico-Experimentales Nro. 003

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Steeven Daniel Pardo Lima Royel Ivan Jima Pardo Mark Anthony Gonzalez Jaramillo Juan Alberto Calopino Martinez Servio Julian Vega Jimenez
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Título de la Práctica	Temporizador de hardware y generación de señal con ESP32
Nombre del Docente	Luis Darío Sinche Cueva
Fecha	Viernes 30 de octubre
Horario	11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Configurar un temporizador de hardware del ESP32 con la API del core 3.x, usar su interrupción para hacer parpadear un LED y generar una onda cuadrada de frecuencia calculada, y medir esa señal con un analizador lógico.

3. Materiales, Reactivos, Equipos y Herramientas

Hardware :

- 1 Módulo ESP32 DevKit V1 de 30 pines (ESP32-WROOM-32) + Cable USB de datos.

- 1 Protoboard.
- 1 LED rojo de 5 mm.
- 1 Resistencia de 330Ω.
- Cables jumper macho-macho.
- Instrumento de laboratorio: Multímetro con modo de frecuencia (Hz) / Osciloscopio.

Software / Entornos virtuales:

- **Wokwi Simulator:** Entorno Web con simulación de ESP32 y Analizador Lógico de 8 canales (wokwi-logic-analyzer).
- **Arduino IDE 2.x:** Con el paquete esp32 by Espressif Systems versión 3.x instalado.
- **PulseView (sigrok):** Software para análisis e importación de capturas de datos .vcd (Value Change Dump).
- **GitHub & EVA UNL:** Para repositorio de código y entrega institucional.

4. Procedimiento / Metodología Ejecutada

Para el desarrollo de la práctica se organizó el trabajo entre los integrantes del grupo, distribuyendo las actividades de simulación, implementación física y verificación de resultados.

El procedimiento se desarrolló mediante las siguientes etapas:

1. Organización y distribución de actividades

Se analizó la guía de la práctica para identificar los objetivos, materiales y procedimientos requeridos. Posteriormente, se distribuyeron las responsabilidades entre los integrantes: unos se encargaron de la simulación en Wokwi, otros del montaje físico del circuito con el ESP32 y, finalmente, se realizó una revisión conjunta del funcionamiento.

2. Configuración y simulación del circuito

Se utilizó Wokwi para implementar el circuito con el ESP32 DevKit V1, se configuraron las conexiones del GPIO19 para el LED y del GPIO18 para generar la señal cuadrada, verificando la correspondencia de los componentes y sus conexiones en el archivo `diagram.json`.

3. Programación del temporizador de hardware

Se implementaron los códigos proporcionados en la guía utilizando la API del ESP32 core 3.x.

En la primera parte se configuró un temporizador con interrupciones para controlar el parpadeo del LED, probando alarmas de 1 000 000, 250 000 y 50 000 microsegundos, además se comprobó el intervalo entre cambios mediante los registros del Monitor Serie.

4. Generación y comprobación de la señal cuadrada

Se configuró el GPIO18 para generar señales de 100, 1000 y 5000 Hz. Para cada frecuencia se calculó el tiempo de alarma mediante la expresión `ALARMA_US = 1000000 / (2 × FREC_SENAL)`. Posteriormente, se compararon los valores calculados con los registros del Monitor Serie a 115200 baudios, verificando la frecuencia mediante el número de cambios del pin.

5. Implementación y verificación física

Los integrantes encargados del montaje conectaron el ESP32 a la protoboard, incorporando el LED y su resistencia. Se revisó la polaridad del LED, las conexiones GPIO y GND y la comunicación USB.

Mediante Arduino IDE se seleccionaron la placa y el puerto serial correspondientes para cargar el programa con una frecuencia configurada de 1000 Hz y por último se verificó la comunicación serial y el comportamiento del LED durante la ejecución.

6. Análisis y comparación de resultados

Se utilizó el analizador lógico de Wokwi para obtener la señal generada. Finalmente, se contrastaron los cálculos teóricos, los registros de la simulación y las observaciones del montaje físico para comprobar el funcionamiento del temporizador y detectar posibles errores de conexión o configuración.

7. Registro de evidencias y documentación

Se recopilaron las capturas de las simulaciones, los registros del Monitor Serie, los códigos utilizados y las evidencias del montaje físico, esto para documentar el procedimiento, presentar los resultados obtenidos y respaldar el cumplimiento de los objetivos de la práctica.

5. Resultados

PARTE A:

En el Código 1 la única línea que cambia entre pruebas es el valor de ALARMA_US. Con timerBegin(1000000) cada tick dura 1 µs, y como la ISR solo levanta la bandera hayAlarma y loop() la baja para cambiar el LED del GPIO19 e imprimir el instante del cambio con millis(), se cumple:

tiempo entre cambios = ALARMA_US × 1 µs

periodo del parpadeo = 2 × tiempo entre cambios

El periodo completo del parpadeo es el doble del tiempo entre cambios porque el LED debe encenderse, apagarse y volver a encenderse (dos cambios) para completar un ciclo. Para cada valor de ALARMA_US se

incluyen el código cambiado, el cálculo manual, la salida del Monitor Serie y la captura de la simulación en Wokwi. El instante de la primera línea impresa varía entre ejecuciones, porque el ESP32 tarda un poco en arrancar y las primeras líneas salen de la ventana del Monitor Serie; por eso no se compara el valor absoluto: lo que se mide (paso A6 y Tabla A) es la diferencia entre dos instantes consecutivos. El circuito no cambia entre pruebas: LED rojo con su resistencia de 330 Ω en el GPIO19.

1. Prueba con ALARMA_US = 1000000

Cálculo manual:

Tiempo entre cambios = 1 000 000 ticks × 1 µs = 1 000 000 µs = **1000 ms**. Periodo completo del parpadeo = 2 × 1000 ms = **2000 ms** (el LED se enciende y se apaga una vez cada 2 segundos).

Salida del Monitor Serie (115200 baudios):

```
entry 0x400805dc
Cambio del LED en t = 1511 ms
Cambio del LED en t = 2511 ms
Cambio del LED en t = 3511 ms
Cambio del LED en t = 4511 ms
Cambio del LED en t = 5511 ms
Cambio del LED en t = 6511 ms
```

Resultados:

La diferencia entre dos instantes consecutivos es 2511 − 1511 = **1000 ms**, igual al tiempo calculado: el LED cambia una vez por segundo. El primer instante (1511 ms) no es 1000 ms porque el arranque y la conexión del Monitor Serie tardan un poco; por eso se compara la diferencia entre líneas y no el valor absoluto.

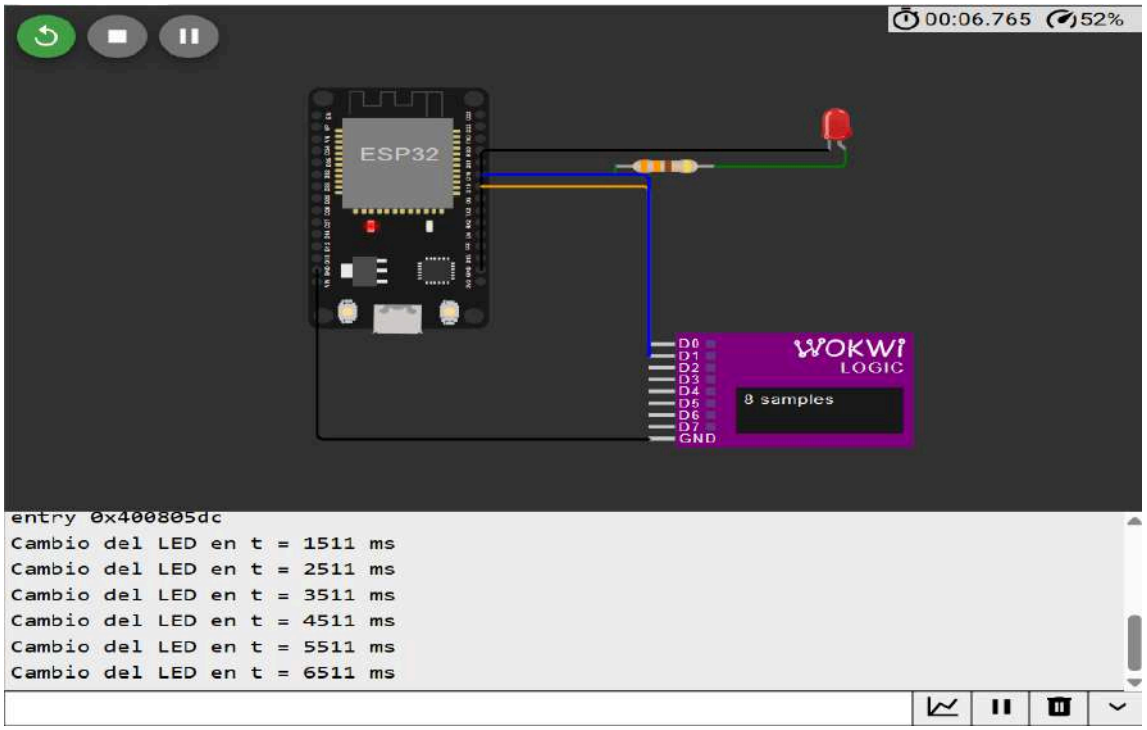


Figura 1. Salida esperada del Monitor Serie con ALARMA_US = 1000000: un cambio cada 1000 ms (periodo completo del parpadeo: 2000 ms).

2. Prueba con ALARMA_US = 250000

Cálculo manual:

Tiempo entre cambios = 250 000 ticks × 1 µs = 250 000 µs = **250 ms**. Periodo completo del parpadeo = 2 × 250 ms = **500 ms** (2 parpadeos por segundo).

Salida del Monitor Serie (115200 baudios):

> Cambio del LED en t = 761 ms
> Cambio del LED en t = 1011 ms
> Cambio del LED en t = 1261 ms
> Cambio del LED en t = 1511 ms
> Cambio del LED en t = 1761 ms
> Cambio del LED en t = 2011 ms
> Cambio del LED en t = 2261 ms

Resultados:

La diferencia entre dos instantes consecutivos es 1011 − 761 = **250 ms**, igual al tiempo calculado: el LED parpadea 2 veces por segundo con un periodo completo de 500 ms.

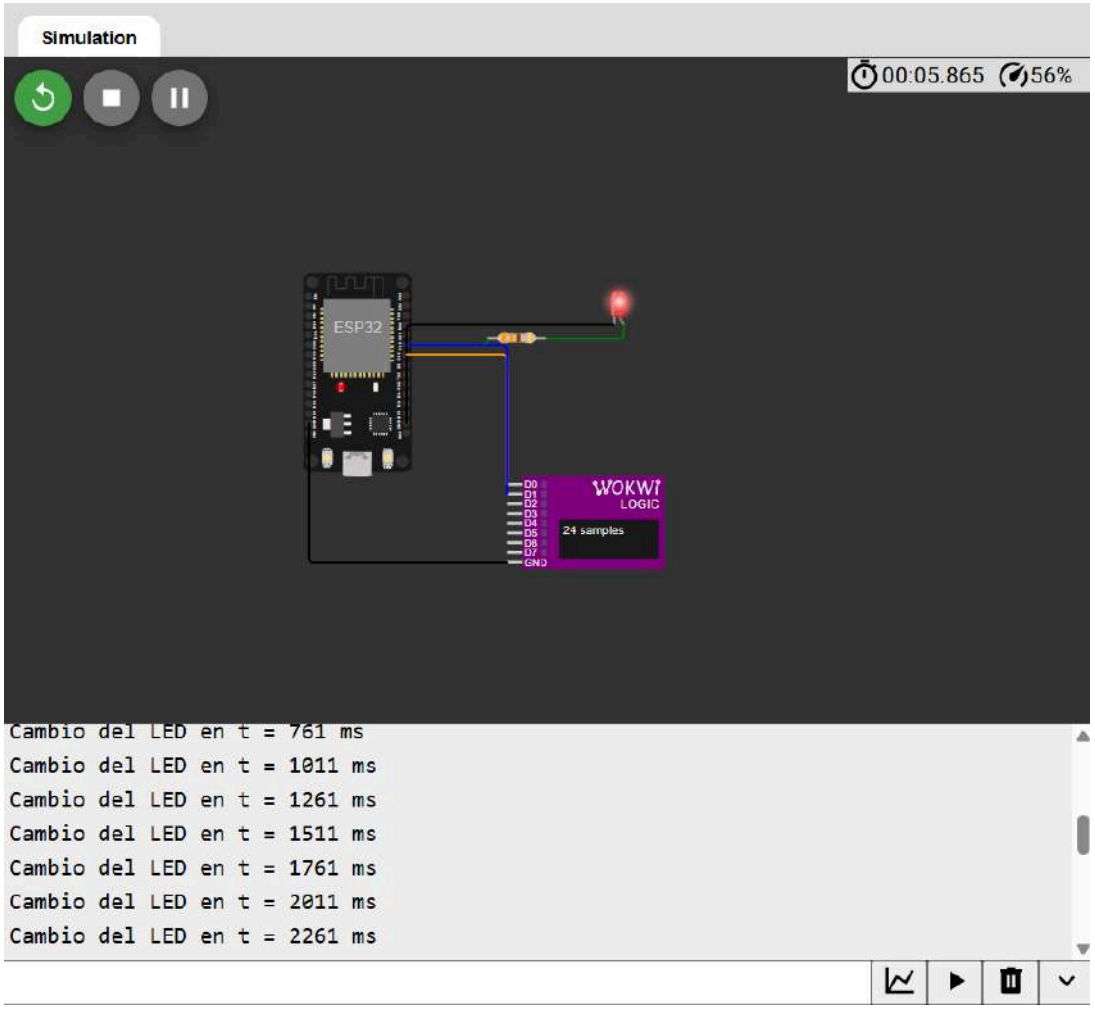


Figura 2. Salida esperada del Monitor Serie con ALARMA_US = 250000: un cambio cada 250 ms (periodo completo del parpadeo: 500 ms).

3. Prueba con ALARMA_US = 50000

Cálculo manual:

Tiempo entre cambios = 50 000 ticks × 1 µs = 50 000 µs = **50 ms**. Periodo completo del parpadeo = 2 × 50 ms = **100 ms** (10 parpadeos por segundo).

Salida del Monitor Serie (115200 baudios):

```
Cambio del LED en t = 559 ms
Cambio del LED en t = 609 ms
Cambio del LED en t = 659 ms
Cambio del LED en t = 709 ms
Cambio del LED en t = 759 ms
Cambio del LED en t = 809 ms
Cambio del LED en t = 859 ms
```

Resultados:

La diferencia entre dos instantes consecutivos es 609 − 559 = **50 ms**, igual al tiempo calculado: el LED parpadea muy rápido, 10 veces por segundo, con un periodo completo de 100 ms.

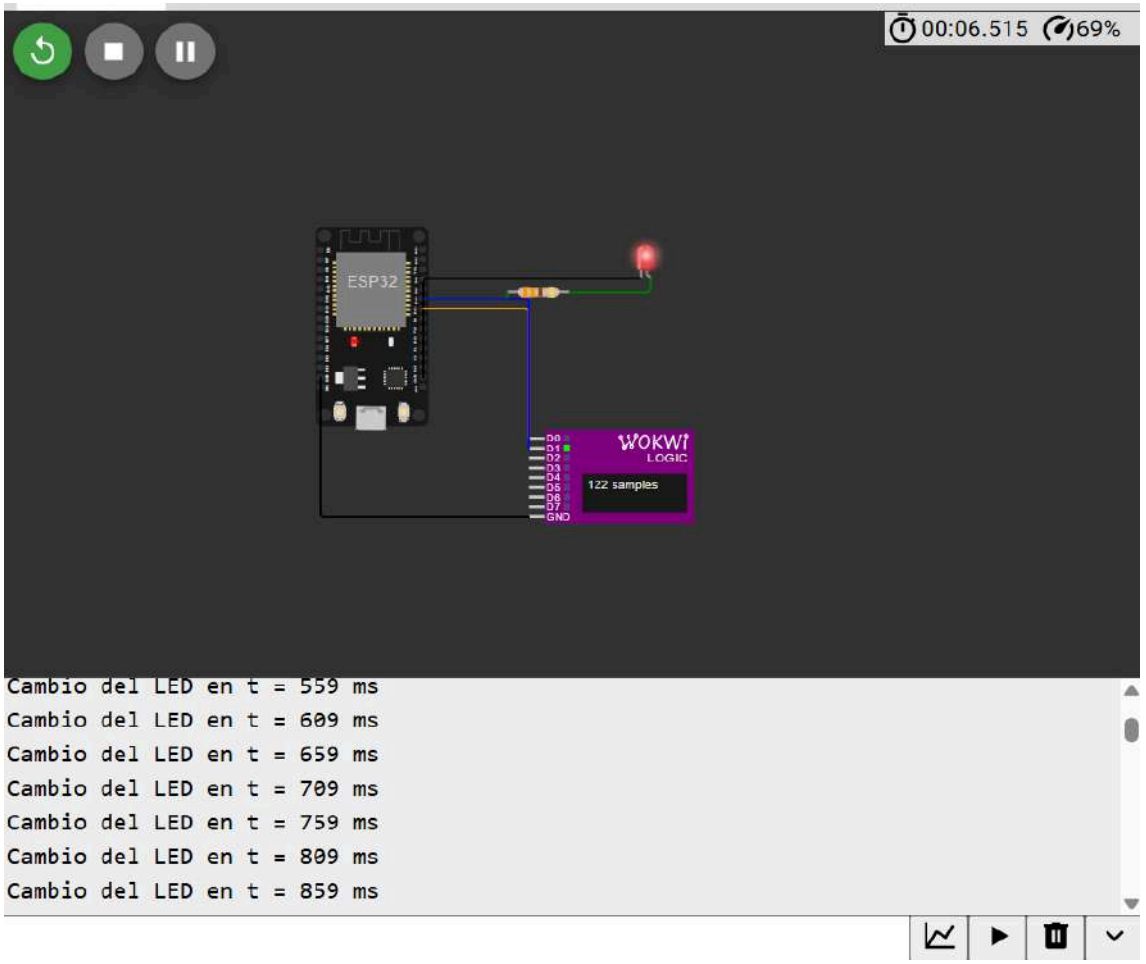


Figura 3. Salida esperada del Monitor Serie con ALARMA_US = 50000: un cambio cada 50 ms (periodo completo del parpadeo: 100 ms).

4. Tabla A. Resultados esperados

Tabla 1. Tiempo entre cambios del LED (Código 1)

ALARMA_US (ticks)	Tiempo calculado entre cambios (ms)	Tiempo medido en el Monitor Serie (ms)	Periodo completo del parpadeo (ms)
1000000	1000	1000	**2000**
250000	250	250	**500**
50000	50	50	**100**

El tiempo medido es la diferencia entre dos instantes consecutivos del Monitor Serie (2511 − 1511 = 1000 ms; 1011 − 761 = 250 ms; 609 − 559 = 50 ms) y coincide con el calculado en las tres pruebas. El instante de la primera línea varía entre ejecuciones por el arranque del ESP32 y la conexión del Monitor Serie; el periodo completo del parpadeo son dos cambios: encendido y apagado.

PARTE B:

En el Código 2 la única línea que cambia entre pruebas es el valor de FREC_SENAL. Como timerBegin(1000000) fija 1 tick = 1 µs y en cada alarma la ISR invierte el GPIO18 (medio periodo de la señal), la alarma se calcula con la fórmula del factor 2 y el pin cambia dos veces por ciclo:

ALARMA_US = 1 000 000 / (2 × FREC_SENAL)

cambios por segundo = 2 × FREC_SENAL

Con estas dos expresiones se obtienen todos los valores esperados: la alarma que imprime el programa al iniciar, los cambios por segundo que reporta el Monitor Serie y la frecuencia medida (cambios / 2). En las tres pruebas el LED de latido del GPIO19 parpadea una vez por segundo. Para cada frecuencia se incluyen el código cambiado, el cálculo manual, la salida esperada del Monitor Serie y la captura de la simulación en Wokwi.

Salida esperada del Monitor Serie (115200 baudios):

> Frecuencia pedida: 100 Hz
> Alarma: 5000 ticks (us)
> Cambios del pin: 200
> Cambios del pin: 400
> Cambios del pin: 600
> Cambios del pin: 800

Resultados:

La alarma impresa (5000 ticks) coincide con la calculada. El contador avanza 200 cambios por segundo, por lo que la frecuencia medida es 200 / 2 = 100 Hz, igual a la pedida.

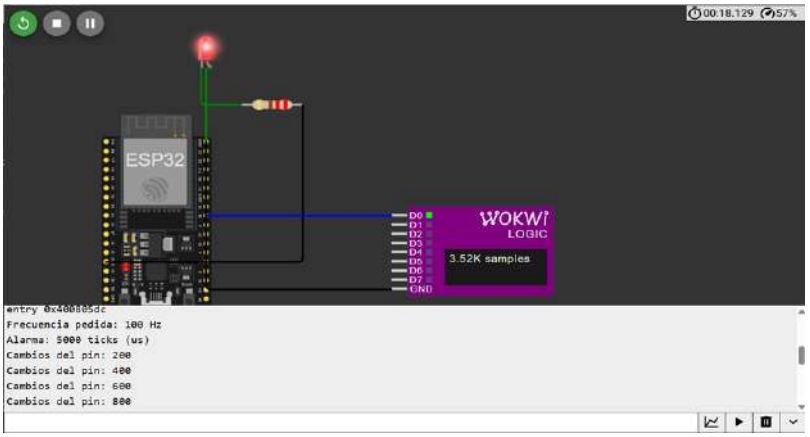


Figura 1. Prueba con FREC_SENAL = 100 Hz en Wokwi: la alarma impresa (5000 ticks) coincide con la calculada y el contador avanza 200 cambios por segundo.

2. Prueba con FREC_SENAL = 1000 Hz

Cálculo manual:

ALARMA_US = 1 000 000 / (2 × 1000) = 500 µs. Periodo: T = 2 × 500 µs = 1000 µs = 1 ms, es decir, f = 1000 Hz. Con esta frecuencia se mide el periodo en PulseView con el analizador lógico (pasos B6 y B7).

Salida esperada del Monitor Serie (115200 baudios):

> Frecuencia pedida: 1000 Hz
> Alarma: 500 ticks (us)
> Cambios del pin: 2000
> Cambios del pin: 4000
> Cambios del pin: 6000
> Cambios del pin: 8000

Resultados:

La alarma impresa (500 ticks) coincide con la calculada. Son 2000 cambios por segundo, así que la frecuencia medida es 2000 / 2 = 1000 Hz. En PulseView, el canal D0 (GPIO18) muestra una onda cuadrada de periodo T = 1,000 ms.

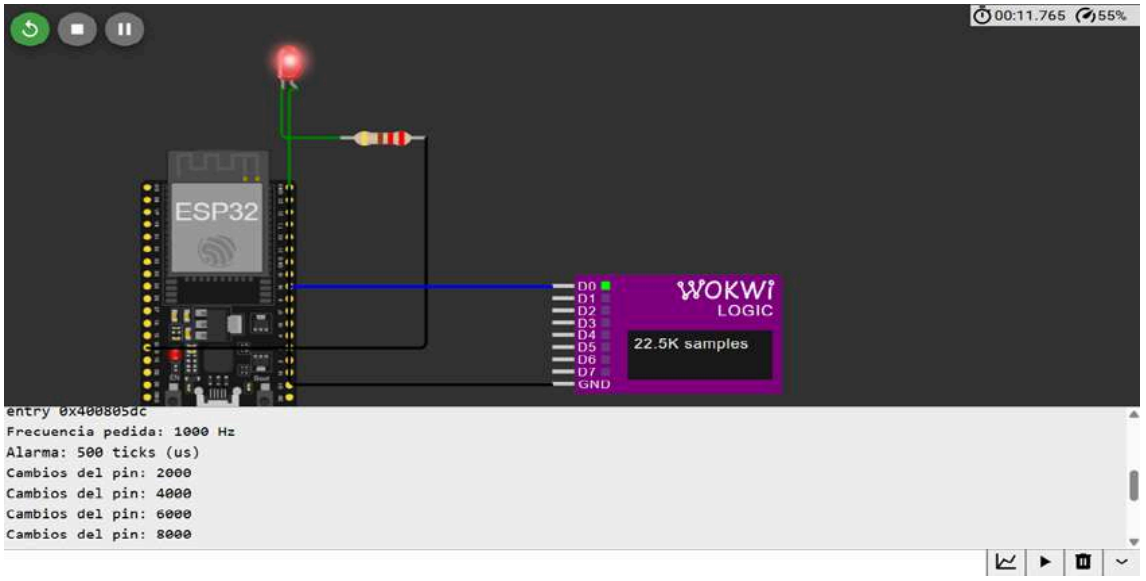


Figura 2. Prueba con FREC_SENAL = 1000 Hz en Wokwi: alarma de 500 ticks y 2000 cambios por segundo; con esta señal se mide el periodo (1,000 ms) en PulseView.

3. Prueba con FREC_SENAL = 5000 Hz

Cálculo manual:

ALARMA_US = 1 000 000 / (2 × 5000) = 100 µs. Periodo: T = 2 × 100 µs = 200 µs = 0,2 ms, es decir, f = 5000 Hz.

> Frecuencia pedida: 5000 Hz
> Alarma: 100 ticks (us)
> Cambios del pin: 10000
> Cambios del pin: 20000
> Cambios del pin: 30000
> Cambios del pin: 40000
> Cambios del pin: 50000

Salida esperada del Monitor Serie (115200 baudios):Resultados:

La alarma impresa (100 ticks) coincide con la calculada. Son 10 000 cambios por segundo, así que la frecuencia medida es 10 000 / 2 = 5000 Hz, igual a la pedida.

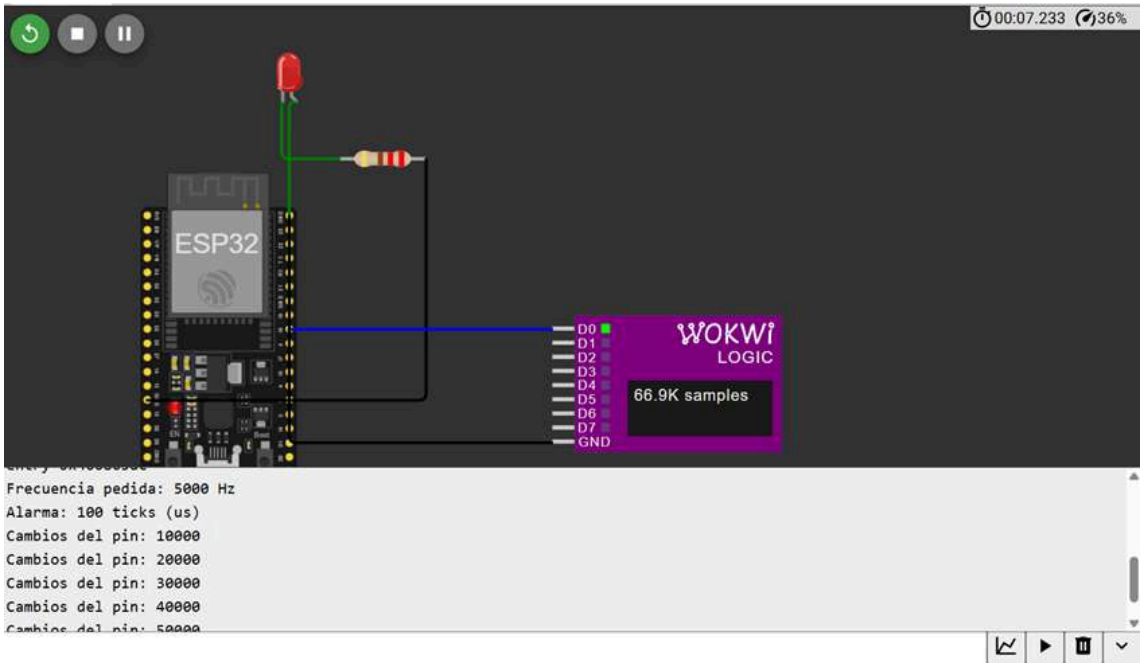


Figura 3. Prueba con FREC_SENAL = 5000 Hz en Wokwi: alarma de 100 ticks y 10 000 cambios por segundo (frecuencia medida 5000 Hz).

Tabla B. Resultados esperados

Tabla 2. Generación y medición de la onda cuadrada (Código 2)

FREC_SENAL pedida (Hz)	ALARMA_US calculada a mano	ALARMA_US impresa por el programa (Monitor Serie)	Cambios por segundo (Monitor Serie)	Frecuencia medida = cambios / 2 (Hz)	Periodo medido en PulseView (ms)
100	5000	5000	200	100	*No se mide*
1000	500	500	2000	1000	1,000
5000	100	100	10000	5000	*No se mide*

ALARMA_US se expresa en ticks de 1 µs (5000 ticks = 5000 µs). El periodo de la señal es el doble de la alarma; por eso, con una alarma de 500 µs la frecuencia es 1000 Hz y no 2000 Hz. El periodo en PulseView solo se mide en la prueba de 1000 Hz (pasos B6 y B7); en las demás pruebas la frecuencia se obtiene con los cambios por segundo.

Parte D. Entrega

Link del repositorio:

https://github.com/ServioVega/APE3_SistemasDigitales.git

6. Preguntas de Control

1. En el Código 1, la ISR solo pone hayAlarma = true. ¿Por qué no se imprime el mensaje directamente dentro de la ISR?

Las ISR deben ser rápidas y deterministas para no bloquear las interrupciones del sistema ni sobrecargar la pila del microcontrolador. Las operaciones de entrada y salida dependen de búferes de transmisión, retardos por baudios e interrupciones del puerto UART, esto causaría desbordamientos, bloqueos críticos del sistema o inconsistencias temporales. Se aplica el patrón bandera: la ISR solo activa la variable y loop() ejecuta la impresión en un tiempo seguro. [1]

2. Con timerBegin(1000000) y ALARMA_US = 250000, ¿cada cuánto cambia el LED? ¿Cuánto dura el periodo completo del parpadeo? Comparen con su Tabla A.

Tiempo entre cambios = 250 000 ticks × 1 µs = 250 000 µs = 250 ms.

El LED conmuta su estado cada 250 ms. Se requiere de dos conmutaciones consecutivas: T = 250 ms + 250 ms = 500 ms. Estos resultados coinciden con las mediciones tabuladas en la Tabla A. [4]

3. Si se quita la palabra volatile de la variable hayAlarma, ¿qué podría pasar y por qué?

El optimizador del compilador asume que las variables que no se modifican dentro del cuerpo de loop() no cambian de valor. Sin la palabra volatile, el compilador guardará el valor de hayAlarma en un registro interno del procesador en lugar de leer la memoria RAM en cada iteración. Lo que pasaría es que loop() no detectaría que la bandera pasó a verdadero, provocando que el código quede atrapado y el LED jamás se conmute. [3, 4]

4. Expliquen con un dibujo por qué, con una alarma cada 500 µs, la señal del GPIO18 tiene 1 kHz y no 2 kHz.

Una onda cuadrada necesita completar un semiciclo alto y un semiciclo bajo para formar un periodo completo [4].

- Alarma 1 (a los 500 µs): Cambia el pin de nivel bajo a nivel alto (Medio periodo = 500 µs).
- Alarma 2 (a los 1000 µs): Cambia el pin de nivel alto a nivel bajo (Medio periodo = 500 µs).

Representación gráfica de la señal:



Figura 4. Resultado de la señal cuadra

Donde el tiempo de alarma es de 500 µs, por lo que el periodo completo es el doble: T_Periodo = 2 × 500 µs = 1000 µs. Calculando la frecuencia: Frecuencia = 1 / T_Periodo = 1 / 0.001 s = 1000 Hz. La frecuencia de la señal es la mitad de la frecuencia de las interrupciones (2 kHz / 2 = 1 kHz).

5. Calculen ALARMA_US para una señal de 2 kHz y para una de 250 Hz.

Usando la fórmula del Factor 2: ALARMA_US = 1 000 000 / (2 * FREC_SENAL) [5]

- Para f = 2 kHz = 2000 Hz:ALARMA_US = 1 000 000 / (2 * 2000) = 1 000 000 / 4000 = 250 µs ticks
- Para f = 250 Hz:ALARMA_US = 1 000 000 / (2 * 250) = 1 000 000 / 500 = 2000 µs ticks

6. ¿Coinciden las frecuencias medidas en la Tabla B con las pedidas? Si hay diferencias, ¿a qué las atribuyen?

Sí, las frecuencias medidas coinciden de forma exacta con los valores teóricos. Esto se da por la generación periódica mediante temporizadores de hardware dedicados. Las mínimas desviaciones que pudiesen presentarse en la prueba física real (Tabla C) se atribuyen a la tolerancia del cristal oscilador de cuarzo físico del ESP32 [5].

7. Con timerBegin(1000000), ¿qué valor de prescaler N calcula el core? Muestren el cálculo a partir del reloj de 80 MHz.

El reloj base del periférico de opera a una frecuencia de f_base = 80 MHz = 80 000 000 Hz. Para lograr una frecuencia de conteo de f_timer = 1 000 000 Hz (donde 1 tick = 1 µs):f_timer = f_base / N --> N = f_base / f_timerN = 80 000 000 Hz / 1 000 000 Hz = 80 Por lo tanto, el core 3.x de Espressif asigna un divisor de prescaler de N = 80. [7]

8. En esta práctica, ¿por qué el contador del temporizador nunca llega a desbordarse?

El contador no llega a desbordar su límite máximo debido a que la función timerAlarm tiene el parámetro de autorrecarga configurado en true. Esto hace que el hardware del temporizador que debe reiniciar su contador de manera automática a cero en el instante en que alcanza el valor del umbral ALARMA_US, repitiendo el ciclo de forma indefinida sin riesgo de desbordamiento. [3]

7. Conclusiones

- El uso de temporizadores de hardware e interrupciones permite medir el tiempo con precisión, sin depender del (loop()). Así se generan señales exactas y estables, evitando los retrasos (jitter) que provocan las funciones bloqueantes como delay().
- La versión 3.x del core del ESP32 facilita la configuración de los temporizadores con timerBegin(1000000). Sin embargo, exige seguir cuidadosamente el patrón de bandera: usar variables de tipo volatile e implementar las rutinas de interrupción (ISR) con el atributo IRAM_ATTR para evitar que el sistema se bloquee.
- La verificación con el analizador lógico simulado en Wokwi, cuyos datos se importaron en PulseView, confirmó la relación matemática del factor 2 en señales periódicas: cada pulso de alarma corresponde a medio periodo de la onda cuadrada generada.

8. Recomendaciones

- Verificar si se tiene acceso a los puertos y descargar la versión correcta del Arduino IDE
- Instalar la libreria corerecta para trabajar con el ESP32

9. Bibliografía / Referencias

[1] L. A. Freire Sánchez, *Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico*. 2023. ISBN: 978-9942-636-22-5.

[2] N. F. Zailani, *Getting Started with Arduino and ESP32: A Beginner's Guide with Wokwi*. Politeknik Mersing, 2024.

[3] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, Tech. Doc. [En línea]. Disponible: https://www.espressif.com/en/support/documents/technical-documents.

[4] Espressif Systems, *Arduino-ESP32 Documentation: Timer API & Core 3.x*, Espressif Systems. [En línea]. Disponible: https://docs.espressif.com/projects/arduino-esp32/.

[5] CodeMagic LTD, *Wokwi Documentation - Logic Analyzer & Simulation*, 2025. [En línea]. Disponible: https://docs.wokwi.com.

[6] sigrok, *PulseView: Logic Analyzer Frontend Documentation*, sigrok.org. [En línea]. Disponible: https://sigrok.org/wiki/PulseView.

10. Anexos

Link de Wokwi Parte A:

https://wokwi.com/projects/479326765080067073

Link de Woki. Parte B:

https://wokwi.com/projects/479322447808414721

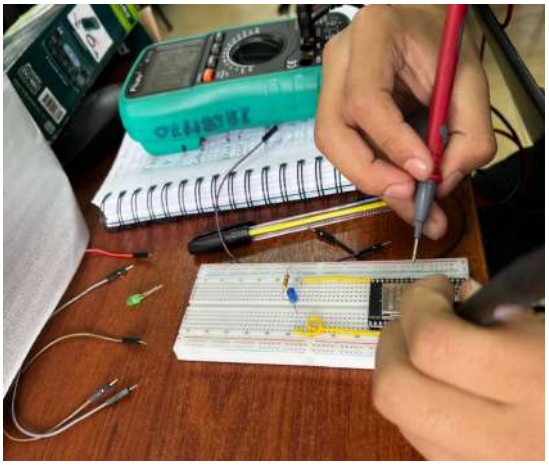
Link del repositorio:

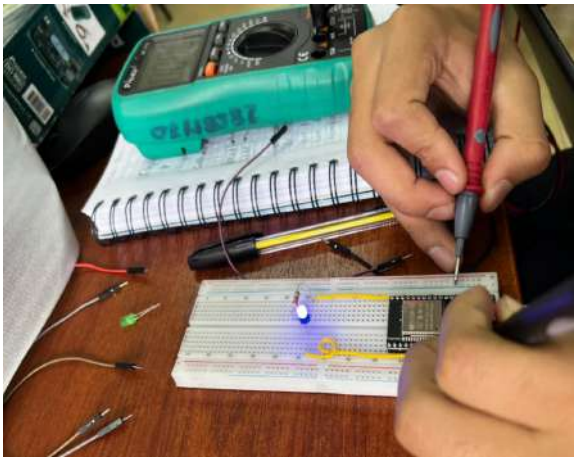
https://github.com/ServioVega/APE3_SistemasDigitales.git

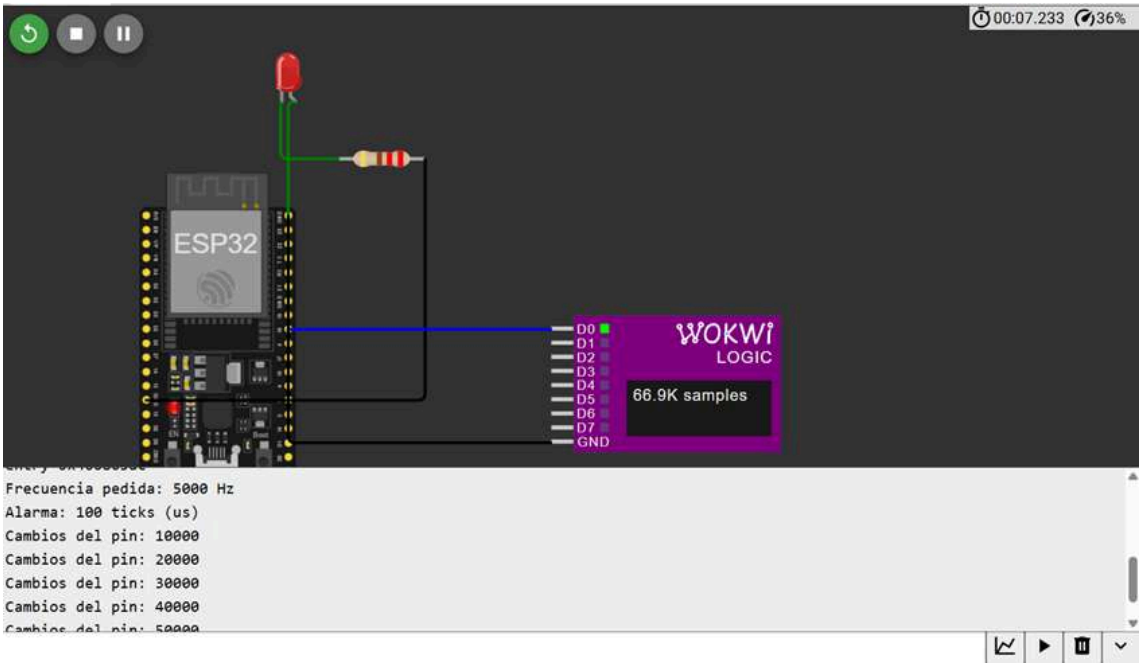
Link del funcionamiento del circuito:

https://drive.google.com/drive/folders/1YqBQbxxaiL8L3Qph2M903tPaeYl8xbt1?usp=sharin

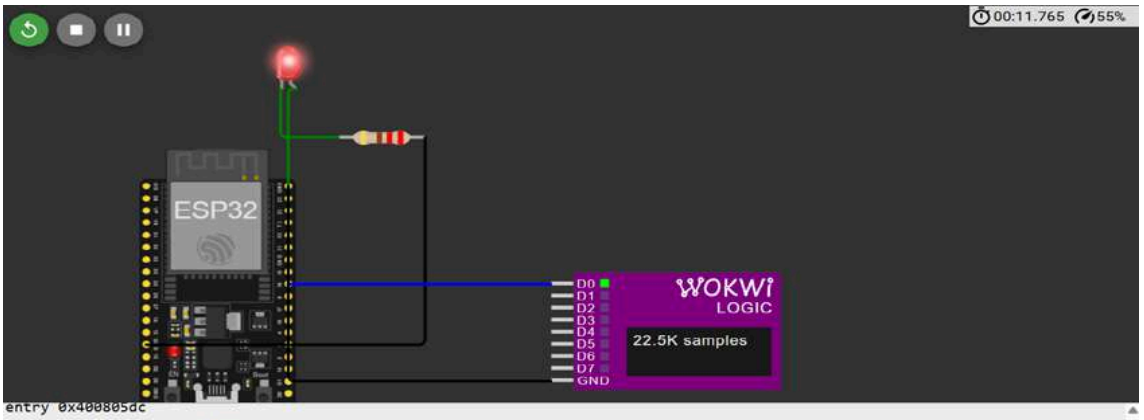
Declaración de uso de IA generativa: Para la elaboración y estructuración del presente informe, se utilizó la herramienta de inteligencia artificial generativa de Google (Gemini) con el fin de guiar la comprensión de los temporizadores de hardware en el core 3.x del ESP32, estructurar la metodología de las pruebas en Wokwi y verificar el razonamiento lógico de las preguntas de control. El uso de la herramienta abarcó principalmente el apoyo en la redacción y organización de las respuestas técnicas, asegurando la consistencia metodológica con las directrices de la guía de la práctica.








```
Frecuencia pedida: 5000 Hz
Alarma: 100 ticks (us)
Cambios del pin: 10000
Cambios del pin: 20000
Cambios del pin: 30000
Cambios del pin: 40000
```




```
entry 0x400805dc
Frecuencia pedida: 1000 Hz
Alarma: 500 ticks (us)
Cambios del pin: 2000
Cambios del pin: 4000
Cambios del pin: 6000
Cambios del pin: 8000
```